

AGÊNCIA DE VOOS



PUC Minas

DISCIPLINA

AED III

ETAPA

Fase II - Estruturas de indexação e ordenação externa

INTEGRANTES

Luiz Henrique de Paula Diniz

Eder Arthur

REPOSITÓRIO DO PROJETO

github.com/LuizDiniz85306/AgenciaDeVoos

Apresentação

Este documento registra as decisões de projeto e descreve a implementação da Fase II da aplicação Agência de Voos. O foco desta etapa é persistir, localizar e ordenar os dados em memória secundária, inclusive após a reinicialização da aplicação.

Objetivos da Fase II

Entrega	Aplicação no projeto
CRUD e relacionamento 1:N	Cliente é o lado 1 e Voo é o lado N; a chave estrangeira (FK) idCliente é armazenada no voo.
Hash Extensível	Busca por chave primária (PK) de Cliente e Voo, com diretório e buckets (compartimentos do índice) persistidos em arquivos binários.
Árvore B+	Consulta por idCliente e recuperação de todos os IDs de voos associados à FK.
Ordenação externa	Geração de sequências ordenadas temporárias (runs) e intercalação em disco por data e horário, preservando o arquivo principal.

Organização do documento

Seção	Conteúdo
1. Modelo de dados	Entidades, chave estrangeira e regras de integridade referencial.
2. Arquitetura	Responsabilidades das telas, serviços, entidades e arquivos binários.
3. Hash Extensível	Persistência do índice de chave primária e manutenção no CRUD.
4. Árvore B+	Consulta por FK, múltiplos valores e estratégia de manutenção da árvore.
5. Ordenação externa	Runs, intercalação de duas vias e arquivo final ordenado.
6. Integridade	Sincronização entre dados e índices e limites da recuperação.

Observação de escopo. A associação legada VooCliente era N:N e foi consolidada em Voo.idCliente. A migração única mantém a primeira associação ativa com ID positivo de cada código de voo, sem apagar o arquivo antigo. A existência do cliente não é revalidada nessa migração.

1. Modelo de dados e relacionamento

As entidades persistidas nesta fase são Cliente e Voo. A chave estrangeira está no lado N: cada voo armazena idCliente. Um cliente pode estar associado a diversos voos sem uma tabela intermediária. O valor -1 representa um voo sem cliente associado.



Um cliente pode ter zero ou vários voos; cada voo referencia zero ou um cliente.

Regras de integridade referencial

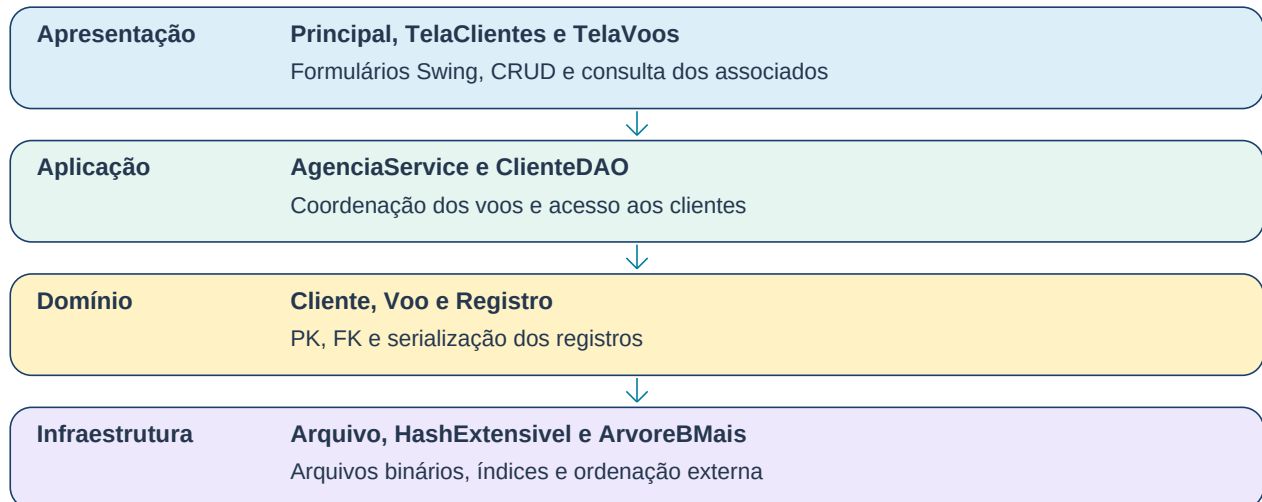
Regra	Comportamento implementado
FK válida	Nas inclusões e alterações feitas por salvarVoo, a FK deve ser -1 ou um ID positivo de cliente existente.
Cliente com voos	TelaClientes bloqueia a exclusão quando a B+ indica voos associados. ClienteDAO não aplica esse bloqueio em chamadas diretas.
Busca dos associados	A consulta por cliente usa a B+ para obter os IDs dos voos e confirma a FK em cada registro recuperado.
Atualização da associação	O ID do voo é removido da lista associada à chave anterior e incluído na lista associada à nova chave da B+.

Decisão 1

O uso de idCliente em Voo materializa o lado N do relacionamento e simplifica a persistência. AgenciaService.salvarVoo valida a existência do cliente mesmo quando chamado sem a interface. Essa validação não abrange a migração legada nem a exclusão direta de clientes pelo DAO.

2. Arquitetura implementada

A solução foi organizada por responsabilidades. As telas apresentam e validam os dados. AgenciaService coordena o CRUD de voos, valida a FK e tenta recuperar a B+ após falhas. Arquivo cuida da persistência binária e do Hash. A Árvore B+ permite consultar os voos associados a um cliente.

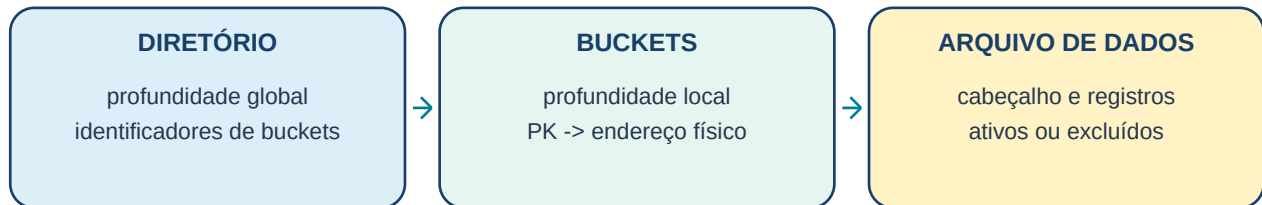


Responsabilidades principais

Componente	Responsabilidade
Principal	Inicializa a aplicação e conecta as telas no mesmo fluxo de navegação.
TelaClientes e TelaVoos	Permitem CRUD, seleção de cliente, consulta de voos, associação e ordenação. TelaClientes verifica vínculos antes de excluir clientes.
AgenciaService	Coordena voos, valida a FK em salvarVoo, consulta a B+ e tenta reconstruí-la após falhas no CRUD.
ClienteDAO	Encaminha o CRUD de clientes para Arquivo, sem verificar diretamente vínculos com voos.
Arquivo	Grava registros binários, gerencia espaços livres, atualiza o Hash por PK e executa a ordenação externa.
HashExtensivel e ArvoreBMais	Mantêm índices em memória durante a execução e persistem suas estruturas em arquivos binários.

3. Hash Extensível persistente

O Hash Extensível é a estrutura de acesso por PK utilizada por Arquivo. Ele associa a chave primária ao endereço físico do registro. O diretório e os buckets são carregados em memória na abertura e persistidos em arquivos, permitindo reutilizar o índice após reiniciar a aplicação.



Busca por ID: a PK determina o bucket; o par armazenado nele fornece o endereço físico do registro.

Arquivos e atualização

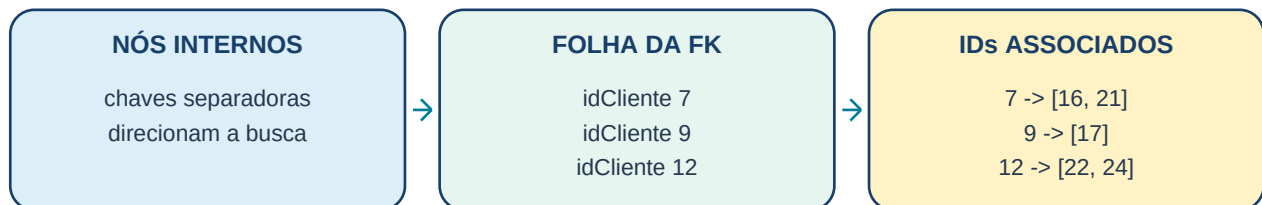
Arquivo ou operação	Conteúdo e comportamento
*.hash.dir.db	Profundidade global e diretório de identificadores dos buckets.
*.hash.buckets.db	Profundidade local e pares PK -> endereço físico dos registros.
Inclusão	A PK e o endereço são inseridos; buckets são divididos quando necessário e o diretório pode aumentar.
Atualização e exclusão	O endereço é atualizado se o registro mudar de posição; exclusões removem a PK do índice.

Decisão 2 - recuperação do Hash

Na abertura, o Hash é carregado dos arquivos binários. A conferência compara os registros ativos, seus endereços e a quantidade de pares com o índice. A reconstrução por varredura ocorre se o carregamento não for aprovado ou se houver divergência detectada. Esse mecanismo pressupõe um arquivo de dados legível; não corrige toda forma de corrupção.

4. Árvore B+ e consulta pela FK

A Árvore B+ secundária é persistida em voos.id_cliente.bplus.db. As folhas armazenam cada chave idCliente e uma lista ordenada de IDs de voos. Os nós internos armazenam chaves separadoras que orientam a busca. A consulta por FK recupera os associados sem varrer todo o arquivo de voos.



Exemplo ilustrativo: cada FK reúne vários IDs; os voos são recuperados pelo Hash de chave primária.

Caminho da consulta 1:N

Etapa	Processamento
1. Informar cliente	O usuário informa o ID do cliente ou seleciona uma linha da lista de clientes cadastrados.
2. Consultar B+	A busca localiza a folha da chave idCliente e devolve todos os IDs de voos associados.
3. Ler os voos	Cada ID retornado é pesquisado pelo Hash do arquivo de voos; a FK do registro também é confirmada.
4. Exibir resultado	A interface apresenta os voos associados no painel Voos do Cliente.

Decisões 3 e 4

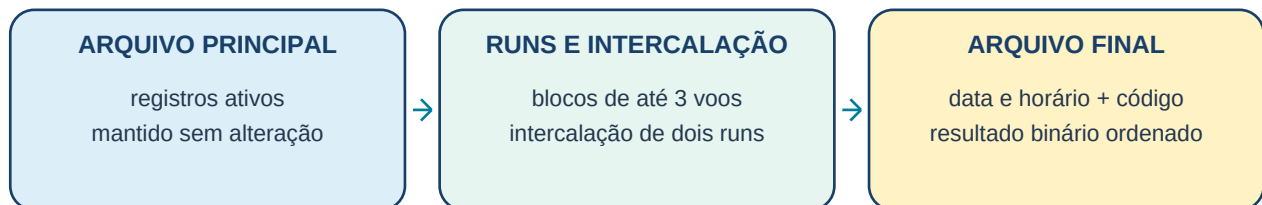
Quando a FK se repete, outro ID de voo é incluído na lista da mesma chave, sem criar uma nova entrada para o cliente. A serialização guarda os nós, as referências aos filhos, os valores das folhas e o encadeamento entre elas.

A estrutura é carregada integralmente em memória. Nesta implementação, cada inserção ou remoção efetiva extrai as entradas, reconstrói toda a B+ e regrava o arquivo. A alteração de um voo associado pode provocar duas reconstruções. Essa estratégia simplifica a manutenção, mas aumenta seu custo conforme a base cresce.

Conferência na abertura. Os pares FK -> IDs extraídos das folhas são comparados com os voos ativos. Essa comparação não constitui uma validação estrutural completa de todos os separadores e encadeamentos.

5. Ordenação externa por intercalação

A ordenação externa usa a data e o horário do voo como chave de ordenação e o código como critério de desempate. A rotina gera sequências ordenadas temporárias (runs) em disco e realiza intercalação de duas vias, mesclando-as aos pares até produzir um único arquivo final.



Resultado: voos_ordenados_por_data_hora.db, separado do arquivo principal.

Decisão 5 - runs e intercalação

Cada bloco contém até três voos, é ordenado em memória e gravado em um run. Nas passagens seguintes, dois runs são intercalados em outro arquivo. A intercalação mantém um registro corrente de cada entrada, sem carregar todos os voos em memória principal. Ao concluir, os registros ativos aparecem em ordem cronológica na saída.

Etapa	Comportamento
Geração dos runs	Lê apenas registros ativos e grava blocos ordenados de até três voos.
Intercalação de duas vias	Lê dois arquivos e grava os registros em ordem em um terceiro arquivo temporário.
Arquivo final	Mantém os IDs dos voos, usa data e horário com código como desempate e não altera o arquivo principal.

Arquivos temporários. O fluxo normal solicita a exclusão dos runs após seu uso. A implementação não garante a limpeza em caso de exceção ou falha na exclusão de arquivos.

6. Sincronização, recuperação e decisões

No fluxo normal do CRUD, a manutenção dos índices acompanha a inclusão, a alteração e a exclusão dos registros. Na abertura, os índices são carregados e conferidos. Após uma exceção no CRUD de voos, AgenciaService tenta reconstruir a B+ a partir dos registros ativos.

Operação	Dados e índices
Criar voo	Arquivo grava o voo e atualiza o Hash. Se houver FK positiva, AgenciaService inclui o ID na lista correspondente da B+.
Alterar voo	Arquivo preserva a PK e atualiza seu endereço no Hash. A B+ remove o ID da lista anterior e o inclui na nova lista quando houver associação.
Excluir voo	Arquivo marca o registro como excluído e remove a PK do Hash; a B+ remove o ID da lista da FK.
Falha no fluxo	O serviço tenta reconstruir a B+ com os voos ativos. Alterações já gravadas não são desfeitas.
Reabertura	Hash e B+ são carregados; se a conferência detectar divergência, o índice correspondente é reconstruído.

Decisão 6 - sincronização e recuperação

A recuperação busca restabelecer a correspondência entre a B+ e os registros ativos. Não há transação nem garantia de gravação atômica entre dados, Hash e B+. Uma falha pode ocorrer depois de os dados terem sido alterados; a própria tentativa de recuperação também pode falhar. O Hash é conferido novamente na abertura.

Decisão 7 - evolução da arquitetura

A principal mudança foi substituir o vínculo N:N legado por idCliente em Voo. Arquivo mantém o Hash por PK, ArvoreBMais persiste o índice de FK e AgenciaService coordena o CRUD de voos e as associações. Essa divisão separa a interface da persistência e permite demonstrar cada estrutura.

Persistência e execução. Os dados e índices ficam em dados e não são versionados. O README descreve a compilação com JDK 21, os arquivos persistidos e o roteiro de demonstração. As classes em bin são geradas pela compilação e precisam existir antes da execução de Principal.